

AsymptoticAnalysis

A current-snapshot technical audit

Source-to-model correctness, native Wolfram compatibility,
and standalone distribution

Repository

<https://github.com/VladimirReshetnikov/Asymptotic>

Pinned revision

6687962f3c858a4f93623cfc496f33e35c6763d4

Exclusion baseline

The consolidated issue register and the finding inventories of all eighteen reports in `external-reports/code-review`.

Principal result

A branch-erasing logarithm rewrite can turn a real, eventually invertible source into the wrong power-log model. Additional findings concern ignored backend defaults, the standalone builder's dependency gate, and native-result diagnostics.

Evidence boundary. Eleven independent Python tests passed, including execution of the exact retrieved Python builder on isolated source fixtures. The mathematical counterexample and its constructive alternative are proved in this article. Native Wolfram execution was unavailable: public Wolfram reproducers, nine regression specifications, and the Wolfram portions of the proposed patches remain unexecuted. No complete upstream-suite pass or native performance measurement is claimed.

Contents

1	Executive assessment	3
1.1	What this audit does not claim	4
1.2	Non-duplication method	4
2	Architecture and actual capability boundaries	4
2.1	The three layers that must remain distinct	4
2.2	Ordinary inversion and exact supports	5
2.3	Exact cores and special-function adapters	5
2.4	Native special functions versus an opaque native result	6
3	Comparison with built-in Wolfram Language	6
3.1	Compare the returned contract, not the spelling of the request	6
3.2	Three concrete comparison patterns	7
3.3	Where the package can be preferable	8
3.4	Where the built-ins remain the natural first tool	8
4	N01: branch erasure in the symbolic-depth parser	9
4.1	The exact source mechanism	9
4.2	Why real-coefficient validation does not save the model	9
4.3	Existence of the actual real inverse	10
4.4	The inverse returned by the changed model	10
4.5	Reproduction candidates and the smallest safe repair	11
4.6	Novelty relative to C07 and C16	11
5	A constructive alternative: retain the winding coefficient	12
5.1	Why a fixed finite Fourier approximation is insufficient	13
5.2	A narrowly scoped implementation opportunity	13
6	N02: declared backend defaults are not consumed	13
6.1	The mismatch	13
6.2	The minimal repair and its boundary	14
6.3	The alias needs an explicit default policy	14
6.4	A precise request-resolution refinement	15
7	N03: the standalone dependency gate is syntax-incomplete	15
7.1	What the builder actually verifies	15
7.2	Executed mutation evidence	16

7.3	The tested conservative repair	16
7.4	Performance and integration	17
8	N04: native-status classification should describe what it knows	17
9	Focused performance, API, and development conclusions	18
9.1	Measure complete semantic jobs	18
9.2	Source identities need local proof obligations	18
9.3	Backend defaults need one owner and one evaluation	18
9.4	Keep new functionality tied to an explicit theorem	19
10	Validation record and how to use the artifacts	19
10.1	Executed work	19
10.2	Proposed source edits	19
10.3	Native execution supplied for a later local run	19
10.4	Practical sequence	20
11	Conclusion	20
A	Novelty crosswalk	21
B	Source-review coverage	22
C	Artifact index	22

1 Executive assessment

The repository is best understood as an asymptotic *modeling and contract* layer, not merely a collection of extra series coefficients. Its specialized inverse engines, explicit positive local coordinates, sparse power-log supports, retained exact cores, source provenance, and selected interval certificates solve problems different from simply printing a Taylor polynomial. The current native bridge further separates package analytic objects from results delegated to `System.Series` or `System.Asymptotic`. That separation is an important improvement over treating every returned expression as an object with the same analytic guarantees.[1, 2, 3, 4]

The most important new concern is earlier in the pipeline than arithmetic on remainders. At the symbolic-depth parser's entry, the source expression is rewritten using

$$\log(u^k) \mapsto k \log u$$

without proving that k is real. A positive u does not make this identity valid for a complex exponent. Worse, a subsequent even power can make all resulting coefficients real, so the recently strengthened real-coefficient checks do not repair the lost branch information. Section 4 gives an explicit real source, proves that its inverse exists near zero, and shows that the source-derived erroneous remainder fails by an exponentially growing factor. The defect is not a request to add general complex asymptotics. It is a request not to silently erase a branch while claiming a real asymptotic model.[2]

The other principal findings are narrower but independently actionable.

ID	Priority	Finding	Evidence and repair status
N01	High	Unguarded logarithm-of-power rewriting in symbolic-depth inversion.	Source proof and independent mathematics; two guard edits proposed; native execution pending.
N02	Medium	Omitted "Backend" is hard-coded as <code>Automatic</code> , ignoring <code>SetOptions</code> .	Complete dispatcher trace; canonical default-resolution edit proposed; native execution pending.
N03	Medium	The standalone dependency gate misses non-bracket application syntax.	Exact retrieved builder executed: eight missed forms; proposed gate rejects all ten dependency forms tested.
N04	Low medium	/"NativeEvaluationStatus" conflates held user data with unresolved native computation.	Classifier inspected; public characterization pending; diagnostic redesign, not a numerical-error claim.

These priorities measure the consequence of the identified mechanism, not its observed incidence in user programs. N01 is a mathematical identity failure in an internal normalization path. N02 is a public option-protocol failure. N03 is a build-time invariant failure under source mutations; it does *not* establish that the current standalone distribution contains an external dependency. N04 is a deliberately lower-confidence public diagnostic concern.[2, 3, 15]

1.1 What this audit does not claim

The Wolfram context connection failed, and a minimal native evaluator request returned HTTP 502. Neither event is evidence of a package failure. There is no native kernel version, package-output transcript, or native benchmark from this audit. Supplied Wolfram expressions are explicitly *reproduction candidates or desired-contract tests*, not copied observations.

Source access used immutable GitHub connector reads. The Python builder was materialized exactly from the returned text and matched its returned Git blob identity. Other package sources were inspected through selected windows or complete small modules, not obtained as a complete executable checkout. The source-coverage appendix distinguishes detailed inspection from architectural review. This article is not a claim that every specialized algorithm or proof in the repository was independently revalidated.

1.2 Non-duplication method

The exclusion baseline is the current consolidated register plus the finding inventories, README finding summaries, or novelty ledgers of all eighteen original reports. This is broader than looking at the old top-level code-review location or assuming an earlier snapshot remains current. The indexed findings are tracked in the supplied machine-readable novelty ledger.[5, 6]

In particular, this report does not present the previously recorded dense native export, logarithmic remainder loss, generic refinement-demand planning, coefficient-equality grouping, parameter-diagonal composition, certificate precision stagnation, or generic native-CI recommendations as fresh discoveries. The new logarithm issue is compared explicitly with C07 and C16; the option issue with D01, C09, and the native-compatibility requirement B01. The distinction is mechanistic, not merely a different example name.

The source now contains fixes discussed by several older reports. For example, the interval-certificate controller has explicit precision escalation logic, and the native result contract refuses ordinary analytic operations on a "Kind"->"Native" object. These are credited as current implementation facts rather than re-reported as missing features. They are not independently certified by a native test run here.[14, 3, 5]

2 Architecture and actual capability boundaries

2.1 The three layers that must remain distinct

A useful implementation model has three layers.

The request/evaluation layer receives held Wolfram syntax, distinguishes package requests from native requests, resolves branches and assumptions, and selects a backend. `NativeCompatibility.wl` deliberately scans option containers rather than interpreting arbitrary rules inside the user's source as wrapper options. It also records held native requests. The source is not just an already-evaluated algebraic tree: delayed options, held expressions, function bodies, and evaluation context can matter.[3, 8]

The asymptotic model layer chooses a positive local coordinate and constructs sparse coefficient blocks. For an ordinary chart this amounts to a finite expression such as

$$s(u) = b + A(u) \sum_{\alpha \in E} u^\alpha P_\alpha(\log u) + O\left(|A(u)| u^\rho (1 + |\log u|)^d\right), \quad u \downarrow 0, \quad (1)$$

with an exact offset b , a retained carrier A , and a finite support E . Different specialized families attach different coefficient algebras or exact cores to this idea. The shared object head does not, by itself, mean that all families are closed under every operation.[2, 9, 10, 13]

The evidence layer records which claims follow from the model and which additionally concern the original source. A formal native order, a Poincaré remainder, a first-neglected-term bound, a numerical consistency check, and an interval existence/uniqueness certificate are different evidence objects. The implementation already reflects parts of this distinction. A complete audit must not silently replace one with another.[3, 12, 11, 14]

N01 attacks the transition from the first layer to the second: once the source has been changed by an invalid identity, correct coefficient algebra on the changed model cannot recover the original function. N02 attacks request resolution. N03 attacks the distribution pipeline that packages these layers. N04 concerns the description of evidence returned by native delegation.

2.2 Ordinary inversion and exact supports

The ordinary inverse constructor identifies a leading term and relative corrections of the form

$$v = au^p \left(1 + \sum_{j=1}^m u^{d_j} Q_j(\log u) \right), \quad d_j > 0. \quad (2)$$

It supports direct Lagrange coefficients, a grouped route, and a Newton route. For a multi-index $\mathbf{k}$, the correction weight is $w = \sum_j k_j d_j$. The code implements the coefficient operator, with $n = \sum_j k_j$, through

$$C_{\mathbf{k}}(L) = \frac{(-1)^n r}{p^n \prod_j k_j!} \prod_{q=1}^{n-1} \left(\frac{d}{dL} + r + w + pq \right) \prod_j Q_j(L)^{k_j}, \quad n > 0. \quad (3)$$

For $n = 0$ the coefficient is 1. This is an implementation-relevant advantage: the package exposes coefficients indexed by exact gaps rather than requiring all exponents to fit one densely materialized rational grid. This discussion credits existing machinery; it is not a renewed claim about the already-known frontier-enumeration bottleneck.[2]

The source distinguishes exponent truncation from marker-depth truncation. Exponent truncation orders actual asymptotic powers. Depth truncation retains a finite multi-index simplex and can admit symbolic exponents whose ordering is proved through assumptions. Those meanings must not be conflated in comparisons with a native integer order. The N01 witness deliberately selects the latter mode, because its dedicated finite parser contains the unsafe rewrite.[2]

2.3 Exact cores and special-function adapters

The repository exposes logarithmic/Lambert, source-coordinate, retained-core, Gamma/Barnes, flat-sector, and Fourier-oriented extensions. Their common purpose is not to expand every nested expression immediately: an appropriate exact outer transformation or leading inverse can be retained, and corrections are computed in a better-conditioned coordinate. The inspected Gamma forward path, for example, combines products in the logarithmic domain and distinguishes exact recurrence simplification from cancellation in a finite Stirling model. The special inverse adapters record both the transformed forward model and its relation to the original function.[1, 7, 10, 11]

The Erfc adapter works with a logarithmic target coordinate and finite tail models, including a first-neglected-term bound on the positive real tail. The Gamma adapter similarly records a finite

Stirling model rather than claiming convergence of that asymptotic series. Lambert and quadratic thresholds use local ramification and an explicit branch choice. These are substantially more informative contracts than a finite expression alone, provided the source-to-model identities and domain hypotheses are valid.[11]

The Zeta and Lerch constructions are a different kind of specialization: they use defining convergent expansions in suitable real regimes and attach explicit value bounds. Native Wolfram also computes overlapping examples; a list of function names therefore does not establish a capability exclusive to this package. The distinction to inspect is the returned representation, bound, coordinate, and reusable provenance. An earlier review already demonstrated native Zeta overlap, so that observation is not counted again here.[12, 19]

2.4 Native special functions versus an opaque native result

There are two distinct uses of native Wolfram computation in the current source. The structured special-function importer calls native `Series` with `Analytic->False`, traverses returned `SeriesData` trees, separates carriers and amplitudes, establishes a real source domain, and transports absolute errors through supported operations. That is an attempt to import native coefficient information into a package analytic model.[13]

By contrast, an explicitly delegated native result is wrapped as `"Kind"->"Native"`. Its complete native result is retained; a package analytic remainder is not fabricated. `requireAnalyticSeries` rejects using that object as though it possessed the ordinary analytic contract. The user may still obtain the native result and perform native formal operations on it. This is a valuable contract barrier.[3, 4]

The first route is an admitted, domain-sensitive importer. The second route is an interoperability envelope. Calling both “native support” without explaining the distinction makes the API look more uniform than its mathematical contracts actually are. The article’s comparison therefore reports them separately.

3 Comparison with built-in Wolfram Language

3.1 Compare the returned contract, not the spelling of the request

Current Wolfram documentation describes `Series` as supporting Taylor, negative and fractional powers, some logarithmic expansions, expansion at infinity, and successive expansion in several variables. It usually returns a `SeriesData` object. It is not accurate to characterize it as an integer-power-only Taylor tool.[21]

`SeriesData` encodes a rational power lattice and permits logarithmic or slower-growing expressions in coefficients; symbolic powers and rapidly growing or oscillating factors may remain outside the object. Native arithmetic, integration, differentiation, composition, and inversion act on that formal representation. This is not the same encoding as a package pair (ρ, d) that explicitly records a logarithmic degree in an analytic remainder.[22]

`Asymptotic` is broader than a request for a rational-lattice series. Its documentation permits automatically inferred scales, complex or infinite centers, and expressions involving integrals, transforms, or differential-equation solutions. Its approximation order is not generally a polynomial degree. A finite native asymptotic answer must be interpreted using that function’s contract, not by attaching an invented package `PowerLogRemainder`. [25]

Task	Built-in Wolfram baseline	Current package role and limitation
Ordinary local series	<code>Series</code> ; native coefficient and formal-order arithmetic.	Adds its own explicit positive chart and remainder metadata; ordinary overlap is substantial.
Power-log, irrational-gap inversion	<code>InverseSeries</code> for native series; <code>AsymptoticSolve</code> for implicit asymptotics.	Dedicated exact-gap and logarithmic coefficient machinery; no blanket claim that native tools fail on every overlapping expression.
General implicit systems	<code>AsymptoticSolve</code> supports systems and multivariate independent variables.	Specialized scalar inverse machinery is not an independently implemented general system solver.
Real branch selection	Native solvers accept real domains, assumptions, and directional specifications.	Positive source charts, source provenance, and explicit inverse-family selection are central package contracts.
Gamma/Erfc and related tails	Built-in functions, <code>Series</code> , <code>Asymptotic</code> , and implicit solution tools overlap many forward cases.	Retained inverse cores, finite-model provenance, and selected transported error bounds are the meaningful additions.
Many other special functions	Native coefficient/asymptotic algorithms are a principal computational source.	Structured importer adds admissibility and envelope handling; opaque delegation adds coverage but not an analytic proof.
Complex and multivariate expansions	Documented native scope is broad.	May be delegated under a native result contract; this does not imply a general complex/multivariate package remainder algebra.
Composition	<code>ComposeSeries</code> requires matching outer center and inner limiting value.	Adds specialized chart and evidence handling, but shared object syntax does not imply universal closure.
Integrals and differential equations	<code>AsymptoticIntegrate</code> and <code>AsymptoticDSolveValue</code> are dedicated engines.	Not independently reimplemented by the inspected scalar inverse layer. Native delegation is not equivalent to an imported analytic certificate.
Numerical inverse checking	<code>FindRoot</code> and arbitrary-precision evaluation offer numerical evidence.	Dedicated checks preserve source intent; supported interval certificates additionally establish bounded claims.

The comparison is grounded in the current package sources and the documented native interfaces; it is not a fresh empirical success/failure matrix.[2, 3, 4, 11, 14, 23, 26, 24, 27, 28]

3.2 Three concrete comparison patterns

An aligned ordinary test. For e^x , a package exclusive cutoff 4 and a native inclusive polynomial order 3 both retain

$$1 + x + \frac{x^2}{2} + \frac{x^3}{6}.$$

The comparison script uses that alignment explicitly:


```
AsymptoticExpansion[Exp[x], {x, 0, 4}, "Backend" -> "Package"]
System`Series[Exp[x], {x, 0, 3}]
```

The coefficient identity follows directly from the exponential series. It is not reported as native output observed during this audit. The request semantics come from the two interfaces.[2, 21]

A logarithmic test. For $\exp(x \log x)$, the same alignment compares retained algebraic powers, but the native displayed order and the package's log-degree metadata need separate examination. Comparing only `Normal[result]` would discard precisely the information being audited. The comparison script records both the finite expression and remainder-related fields, rather than treating equality after `Normal` as sufficient. This follows from the representations' definitions, not from a newly asserted failure in the old export issue.[2, 22]

An irrational-gap inverse. For

$$y = x + x^{1+\sqrt{2}}, \quad x \downarrow 0,$$

the dedicated inverse route and a real `AsymptoticSolve` request address the same source equation. Their order arguments need not mean the same thing. One should compare actual exponents, the selected root, the residual against the original equation, and the remainder meaning. The supplied requests are coverage probes; no result or speed ranking is claimed. Native `AsymptoticSolve` explicitly supports real solutions, implicit functions, systems, and automatically inferred scales.[2, 26]

3.3 Where the package can be preferable

The package is particularly attractive when the consumer needs more than a finite formula: an exact sparse support, a named inverse branch, an exact logarithmic or Lambert core, explicit input-model truncation, subsequent precision-aware operations, or a source-linked certificate. Those features should be assessed by checking that their metadata really describes the expression and source at hand. They do not become valid merely because the object carries an `Assumptions` key.[2, 11, 14]

A strong implementation strategy is consequently to use native Wolfram as a coefficient or solution oracle where suitable, while keeping branch admission, remainder import, source provenance, and supported operations explicit. The current separation of structured import and opaque native delegation follows that strategy in part. N01 demonstrates why branch checks must precede coefficient-domain checks; N02 demonstrates why request defaults must be resolved before choosing which contract will be returned.[3, 13, 2]

3.4 Where the built-ins remain the natural first tool

For ordinary formal power-series manipulation, native `InverseSeries` and `ComposeSeries` are direct tools. The former inverts an appropriate native series; the latter composes series with compatible centers. Native `SeriesData` arithmetic already propagates its formal truncation order. A wrapper is not automatically a better replacement for those operations. [23, 24, 22]

For asymptotic integrals or differential equations, Wolfram exposes dedicated engines rather than requiring a prior scalar inverse reduction. `AsymptoticIntegrate` accepts integral problems; `AsymptoticDSolveValue` accepts equations, systems, and parameter-asymptotic forms. These capabilities are relevant comparison baselines, not a demand that the package independently reimplement them. The broad wishlist is already in the older reports and is not repeated as a new development finding.[27, 28, 5]

4 N01: branch erasure in the symbolic-depth parser

4.1 The exact source mechanism

The relevant source is `parseFinite` in `src/Kernel/AsymptoticAnalysis.wl`, lines 916–928. Line 917 contains two rewrite rules; formatted separately, they are:

```
Log[u^k_] /; FreeQ[k, u] :> k Log[u]

Log[c_ u^k_] /;
  FreeQ[c, u] && FreeQ[k, u] &&
  TrueQ[Simplify[c > 0, ass]] :> Log[c] + k Log[u]
```

The exponent is required to be independent of u , but not real. The second rule proves positivity of the multiplicative coefficient c but again does not establish realness of k . These rules execute before the logarithm is replaced by the formal polynomial variable and before `rowsToModel` checks real coefficients.[2]

For a positive real base, the standard identity is safe for a real exponent. For $k = a + ib$, however,

$$\text{Log}(u^k) = a \log u + i \text{Arg}\left(e^{ib \log u}\right), \quad u > 0, \quad (4)$$

whereas the rewrite produces $a \log u + ib \log u$. The difference is an integer multiple of $2\pi i$, generally depending on u . Wolfram’s documentation explicitly warns that unrestricted power expansion ignores branch information and need not preserve numerical values. The package rule performs one of those branch-sensitive transformations without its missing hypothesis.[30, 29]

Proposition 4.1. *For a fixed $k \in \mathbb{C}$ with $\text{Im } k \neq 0$, the identity $\text{Log}(u^k) = k \log u$ does not hold throughout any punctured positive interval $(0, \varepsilon)$.*

Proof. Write $b = \text{Im } k$. The imaginary part of the principal logarithm in (4) remains in a bounded interval of length 2π . The imaginary part of $k \log u$ is $b \log u$, which is unbounded as $u \downarrow 0$. Equality on an entire sufficiently small positive interval is therefore impossible. $\square$

This is stronger than exhibiting a branch-cut exception far from the expansion point. No amount of restricting the positive source variable to a smaller neighborhood makes this particular rewrite eventually valid when $\text{Im } k \neq 0$.

4.2 Why real-coefficient validation does not save the model

Consider the parameterized input

$$f(x) = x + x^2 \text{Log}(x^a)^2, \quad a^2 = -1, \quad x > 0. \quad (5)$$

The expression contains a symbolic parameter a , not a literal `Complex` atom. Its source syntax passes the core’s syntactic exactness condition. After the unsafe rewrite it becomes

$$\tilde{f}(x) = x + a^2 x^2 (\log x)^2 = x - x^2 (\log x)^2.$$

All coefficients of this changed power-log model are real under the stated assumption. Thus a gate that validates only the resulting coefficients can approve the wrong model.[2]

The two possible parameter values $a = i$ and $a = -i$ produce the *same real* source in (5). Define

$$\theta(t) = \text{Arg}(e^{it}), \quad P(t) = \theta(t)^2. \quad (6)$$

At the two choices of principal endpoint, the square has the same value. Consequently P is continuous and 2π -periodic, and

$$f(x) = x - x^2 P(\log x), \quad 0 \leq P(t) \leq \pi^2. \quad (7)$$

The actual perturbation is bounded by $\pi^2 x^2$. The rewritten perturbation is $x^2(\log x)^2$, which has a different leading amplitude. This is not a merely complex-valued input outside a real-only output contract: the source itself is real on the entire positive approach.

4.3 Existence of the actual real inverse

On every interval away from the phase corners, $P'(t) = 2\theta(t)$, so $|P'| \leq 2\pi$. Continuity across the corners makes P globally Lipschitz with constant 2π . Set

$$\delta = \frac{1}{4(\pi^2 + \pi)}. \quad (8)$$

Where f is differentiable,

$$f'(x) = 1 - 2x(\theta(\log x)^2 + \theta(\log x)) \geq 1 - 2x(\pi^2 + \pi) \geq \frac{1}{2}, \quad 0 < x \leq \delta.$$

There are finitely many corners on any compact subinterval of $(0, \delta]$. Continuity and the derivative lower bound on the pieces therefore imply strict increase on that interval. Also $f(x) \rightarrow 0$ as $x \downarrow 0$. Thus a unique real inverse g exists on $(0, f(\delta))$.

The source is not asserted to be an analytic power-log germ: its derivative has phase corners accumulating at zero. That is precisely why a sound parser must either retain the periodic coefficient in an admitted representation or reject the unsupported normalization. It must not replace it by an unrelated analytic-looking model.

4.4 The inverse returned by the changed model

For the changed model $\tilde{f}(x) = x - x^2(\log x)^2$, the depth-one Lagrange construction has leading power $p = 1$, one gap $d_1 = 1$, and $Q_1(L) = -L^2$. Formula (3) gives the first correction L^2 . The depth remainder formula in construct assigns power 3 and logarithmic degree 4. Thus the source-derived core result is

$$\tilde{g}_1(y) = y + y^2(\log y)^2, \quad \mathcal{O}(y^3(1 + |\log y|)^4). \quad (9)$$

That is a valid type of approximation for the *changed* source, but not for (7). This algebraic trace is not a captured public Wolfram output; the supplied probes separate private normalization, core construction, and the complete public facade.[2]

Choose the exact sequence

$$x_n = y_n = e^{-2\pi n}, \quad n = 1, 2, \dots$$

Then $x_n^{\pm i} = 1$, so $P(\log x_n) = 0$, $f(x_n) = x_n$, and $g(y_n) = y_n$. The error in (9) is exactly $y_n^2(2\pi n)^2$. Its ratio to the claimed remainder scale is

$$\frac{|\tilde{g}_1(y_n) - g(y_n)|}{y_n^3(1 + |\log y_n|)^4} = \frac{e^{2\pi n}(2\pi n)^2}{(1 + 2\pi n)^4} \rightarrow \infty. \quad (10)$$

This proves failure of the advertised asymptotic scale for the original source; no fitted constants or numerical root solver are needed for the conclusion.

n	y_n	Ratio in (10)
1	$1.86744 \cdot 10^{-3}$	7.51324
2	$3.48734 \cdot 10^{-6}$	$1.33681 \cdot 10^3$
3	$6.51241 \cdot 10^{-9}$	$3.51445 \cdot 10^5$
5	$2.27110 \cdot 10^{-14}$	$3.93577 \cdot 10^{10}$
10	$5.15790 \cdot 10^{-28}$	$4.61038 \cdot 10^{23}$
20	$2.66039 \cdot 10^{-55}$	$2.30603 \cdot 10^{50}$

The displayed values were independently evaluated with mpmath. They illustrate the exact formula, rather than serving as a substitute for its proof.

4.5 Reproduction candidates and the smallest safe repair

The public candidate is:

```
Clear[a, x, y];
AsymptoticInverse[
  x + x^2 Log[x^a]^2,
  {x, 0}, {y, 1},
  Assumptions -> a^2 == -1,
  "Truncation" -> "Depth"]
```

For localization, the characterization script additionally calls `parseFinite` directly and invokes the core constructor under the package's failure catcher. These distinguish an identity defect in the private parser from any earlier public-family routing or rejection. A future native run should record all three, rather than declaring the public result solely from this source trace.

Both rewrite rules should require

```
TrueQ[Simplify[Element[k, Reals], ass]]
```

in addition to their existing conditions. For a fixed real k and positive u , u^k is positive, and the identity is valid. In the scaled rule, the existing proof $c > 0$ completes the sufficient conditions. An unproved condition should leave the logarithm unnormalized; this finite power-log parser will then reject it rather than fabricate coefficients.

This is an intentionally conservative patch. It does not prove that every normalization in every other engine is sound, and it does not introduce a complex-asymptotic mode. It also does not convert an inconclusive proof into "false": it simply refuses to use the identity without a sufficient proof. The proposed edit tool patches exactly these two anchors, and the Wolfram tests check both the rejection case and a real-exponent positive control.

4.6 Novelty relative to C07 and C16

C07 concerns admission of nonreal coefficients into a purportedly real result. Here the coefficients *after rewriting* really are real. The failed obligation is equality between the source and the normalized expression. C16 concerns treating formal Taylor information for an opaque source as analytic information. Here the source is explicit and the failure is a specific branch identity, not an unknown derivative or unjustified smoothness assumption. Neither older repair, by itself, addresses this mechanism.[5, 18, 20]

The appropriate invariant is therefore a chain:

$$\text{valid source identity} \implies \text{admitted coefficient model} \implies \text{valid remainder algebra.}$$

A check at the second or third stage cannot compensate for a failed implication at the first.

5 A constructive alternative: retain the winding coefficient

Rejecting the unsafe rewrite is the immediate correctness repair. The same example also identifies a concrete extension beyond rejection: retain the bounded periodic function $P(\log y)$ instead of expanding it as an unbounded polynomial in $\log y$.

Proposition 5.1. *For the real source (7), its inverse satisfies*

$$g(y) = y + y^2 P(\log y) + O(y^3), \quad y \downarrow 0. \quad (11)$$

More explicitly, for $0 < y < 3\delta/4$,

$$|g(y) - (y + y^2 P(\log y))| \leq Ky^3, \quad K = \frac{112}{27}\pi^4 + \frac{32}{9}\pi^3 \approx 514.311879893224285, \quad (12)$$

where δ is given by (8).

Proof. Let $x = g(y)$. Since $x \leq \delta$ and $P \leq \pi^2$,

$$y = x - x^2 P(\log x) \geq \frac{3}{4}x.$$

The assumed bound on y ensures $x < \delta$, because $f(\delta) \geq 3\delta/4$. Therefore

$$y \leq x \leq \frac{4}{3}y, \quad 0 \leq x - y \leq \pi^2 x^2 \leq \frac{16}{9}\pi^2 y^2.$$

Using the 2π Lipschitz constant of P ,

$$\begin{aligned} |x - y - y^2 P(\log y)| &= |x^2 P(\log x) - y^2 P(\log y)| \\ &\leq \pi^2 (x - y)(x + y) + 2\pi y^2 \log(x/y) \\ &\leq \pi^2 \left(\frac{16}{9}\pi^2 y^2 \right) \left(\frac{7}{3}y \right) + 2\pi y^2 \left(\frac{16}{9}\pi^2 y \right), \end{aligned}$$

where $\log(x/y) \leq (x - y)/y$ was used. The resulting constant is exactly K . □

The independent checks evaluated this bound at sixty targets with high-precision bisection. Those tests are useful implementation sanity checks, but the uniform analytic conclusion is the proposition, not the floating-point sample. A separate Wolfram reference file returns the approximation and its conditional bound as ordinary expressions. It does not claim to install a new package coefficient algebra or to generate an InverseCertificate.

5.1 Why a fixed finite Fourier approximation is insufficient

On $[-\pi, \pi]$, the coefficient is $P(t) = t^2$, periodically continued. Direct integration of its Fourier coefficients gives

$$P(t) = \frac{\pi^2}{3} + 4 \sum_{m=1}^{\infty} \frac{(-1)^m \cos(mt)}{m^2}. \quad (13)$$

The series is absolutely and uniformly convergent. If P_M is its ordinary partial sum, then

$$\|P - P_M\|_{\infty} = 4 \sum_{m>M} \frac{1}{m^2} \asymp \frac{4}{M}.$$

The upper bound follows by the triangle inequality, and equality is attained at $t = \pi$, where the terms all have the same sign. Replacing P by P_M in (11) introduces an error of order y^2/M . Consequently, to preserve a *uniform* cubic bound using this partial-sum strategy, one needs M of order at least $1/y$. A fixed number of harmonics does not preserve the next asymptotic order.

This is a specific representation-cost argument, not a benchmark of the repository's Fourier implementation and not a lower bound for every possible representation of P . In fact, retaining P exactly as a periodic piecewise quadratic coefficient has constant structural size. It provides a concrete reason to support a bounded coefficient with a Lipschitz contract instead of forcing every such coefficient into a fixed finite harmonic polynomial.

5.2 A narrowly scoped implementation opportunity

A prototype coefficient object could record a held function, its period, a supremum bound, a Lipschitz constant, and the domain of its parameters. For this example the data are

$$(P, 2\pi, \pi^2, 2\pi).$$

The first supported operation need only be the one-step inverse estimate used in Proposition 5.1. It requires a value bound and a Lipschitz bound, not analyticity or an unrestricted derivative series. This is narrower and more testable than announcing a general transseries extension.

For nonnegative bounded Lipschitz P with $\|P\|_{\infty} \leq M$ and Lipschitz constant L , where $M + L > 0$, the same proof works after taking $\delta \leq 1/[2(2M + L)]$. Its cubic error constant can be taken as

$$\frac{112}{27}M^2 + \frac{16}{9}ML.$$

A useful implementation milestone is therefore an explicit, small theorem-backed constructor for $x - x^2P(\log x)$, with exact coefficient retention and no claimed higher derivative regularity. General closure, higher-order corner behavior, and integration remain separate tasks. This proposal is derived from N01, not an unimplemented feature asserted to exist today.

6 N02: declared backend defaults are not consumed

6.1 The mismatch

`NativeCompatibility.wl` merges a "Backend" $\rightarrow$ Automatic entry and native option lists into `Options[AsymptoticExpansion]`. It also copies those options onto the short alias `AsymptoticExpand`. The dispatcher then collects explicit selector rules from held trailing option containers. In the absence of such a rule, it does not consult the declared defaults:

```
backend = If[values === {}, Automatic, ReleaseHold[First[values]]];
```

Thus the option table and the dispatch implementation are disconnected for this key.[3]

Wolfram's `SetOptions` interface changes the default option settings associated with a symbol. A function that advertises configurable options normally obtains its effective values through that option protocol. The current dispatcher instead interprets “not explicitly supplied” as “always Automatic.” [31]

A source-derived public witness is:

```
saved = Options[AsymptoticExpansion];
Internal`WithLocalSettings[
  SetOptions[AsymptoticExpansion, "Backend" -> "Series"],
  s = AsymptoticExpansion[Exp[x], {x, 0, 3}];
  {s["Kind"], s["NativeBackend"], Normal[s]},
  Options[AsymptoticExpansion] = saved
]
```

The expected option-protocol behavior is a native `Series` result. The inspected source instead selects `Automatic`; for this ordinary exact request, the package path is the natural successful route. The resulting order convention can differ as well: package cutoff 3 omits the cubic term, whereas native `Series` order 3 retains it. The exact public result remains a native reproduction candidate, not an observation made here.[2, 3, 21]

6.2 The minimal repair and its boundary

The narrow replacement is:

```
backend = If[values === {},
  OptionValue[AsymptoticExpansion, {}, "Backend"],
  ReleaseHold[First[values]]];
```

This preserves the explicit selector handling and resolves the canonical default only when no explicit selector exists. The test specifications cover a constant default, an explicit override, a delayed default with a counter, the alias under a canonical default, and the unchanged explicit-native control.

The patch does not silently invent a policy for every merged option. Native options on the wrapper, native backend defaults, package defaults, and explicit package-only protections are separate concerns. For example, blindly passing *all* defaults into the existing automatic-protection logic could mark nearly every request as package-only merely because “`MaxTerms`” has a default value. The dispatcher must know both a value and its origin. [3, 4]

6.3 The alias needs an explicit default policy

The source defines

```
Options[AsymptoticExpand] = Options[AsymptoticExpansion];
AsymptoticExpand[args___] := AsymptoticExpansion[args];
```

The alias's own option table is not read by the delegation. The minimal patch makes a canonical default work through the alias, but changing the alias's *own* defaults still does not automatically

change canonical dispatch. This should not be left to users to infer from copied Options output. [3]

There are two coherent designs. A true synonym can document and expose one canonical configuration owner. Alternatively, the alias can be an independent entry point whose defaults are resolved first and then passed to a shared implementation. The latter must preserve the distinction between an explicit caller request and a wrapper-resolved default. Copying an option list once while ignoring it at runtime is neither design.

6.4 A precise request-resolution refinement

The earlier reports already recommend typed requests and clearer cutoff semantics. This report does not repeat that general recommendation as new. The concrete delta is to retain the *origin* of each resolved selector:

```
<|"Value" -> "Series",
  "Origin" -> "CanonicalDefault",
  "Owner" -> AsymptoticExpansion,
  "Evaluated" -> True|>
```

An explicit selector has origin "Caller"; an inherited native default is another case. Backend selection consumes a resolved value once. Protection logic can then deliberately inspect caller intent rather than accidentally using the presence of every effective default as a veto.

A useful invariant is that the returned object records the backend actually chosen and the effective order meaning. A useful test is that delayed defaults are not re-evaluated solely for metadata. Neither requirement demands that all native and package semantics become identical. It demands that dispatch be explainable and consistent with the advertised option protocol.

7 N03: the standalone dependency gate is syntax-incomplete

7.1 What the builder actually verifies

The Python builder recursively inlines recognized companion loads into one standalone Wolfram file. It masks strings and nested comments, checks remaining executable text for forbidden dependency operations or runtime file-location symbols, and emits deterministic source boundaries. The dedicated upstream Python tests inspect load order, missing companions, cycles, inert text, freshness, and several bracket-form dependencies. These are useful protections. [15, 16]

The incomplete part is the dependency expression:

```
(?:Get|Needs|Import|OpenRead|ReadList|Read|BinaryRead|
  URLRead|URLExecute|URLDownload)\s*\[
```

It recognizes selected heads only when followed by a bracket. Wolfram source also admits prefix, postfix, and higher-order application forms. Thus `Get@"missing.wl"`, `"missing.wl"//Get`, and `Apply[Get, {"missing.wl"}]` are not recognized by that gate. The preceding companion-load inliner also does not rewrite these forms.[15, 32, 33]

7.2 Executed mutation evidence

The exact retrieved builder was loaded as a Python module. Each test created a temporary miniature `src/Kernel` tree with a valid entry-point directory setup, inserted one candidate statement, and called the builder's actual `assemble()` function. No Wolfram fixture content was executed and no actual network request was made.

Inserted executable form	Baseline	Proposed gate
<code>Get["missing.wl"]</code>	Reject	Reject
<code>System'Get["missing.wl"]</code>	Reject	Reject
<code>Get@"missing.wl"</code>	Accept	Reject
<code>"missing.wl"//Get</code>	Accept	Reject
<code>Import@"missing.json"</code>	Accept	Reject
<code>"Missing'"//Needs</code>	Accept	Reject
<code>Apply[Get,{"missing.wl"}]</code>	Accept	Reject
<code>System'Get@"missing.wl"</code>	Accept	Reject
<code>ReadList@"missing.wl"</code>	Accept	Reject
<code>URLDownload@"https://example.invalid/x"</code>	Accept	Reject

The four controls cover quoted text, nested comments, unrelated larger symbols including `Get$Helper`, and an ordinary function definition. Both versions accept them. The proposed version also preserves the intended direct companion inlining. The complete independent suite contains eleven test methods and additional subcases; its actual log and structured mutation results are supplied.

This proves a packaging-invariant gap under concrete source edits. It does not prove that an existing module uses any missed form, and it is not an allegation of an external load hidden in the current distribution. The observed failure is that the advertised gate would allow such an accidental dependency to be introduced unnoticed.

7.3 The tested conservative repair

After comment and string masking, the proposed gate recognizes the forbidden *symbol tokens* regardless of how they are applied. Identifier boundaries must include `$` as well as ordinary word characters, so names such as `Get$Helper` are not mistaken for `Get`. A context-qualified occurrence such as `System'Get` remains detectable.

This deliberately conservative policy can reject a forbidden head stored as held data, or a same-leaf-name symbol from another context. Such a rejection is a request for source review, not a claim that the occurrence necessarily executes I/O. That is an acceptable tradeoff for a small self-contained distribution builder, provided the policy is stated. The existing gate was already conservative about bracket-form calls inside held code.[15]

The repair is not a security sandbox and does not decide arbitrary program behavior. A dynamically constructed function name or other reflective mechanism can evade a literal-token check. Proving absence of all possible runtime I/O in an unrestricted symbolic language is not the builder's present contract. The specific implementable requirement is to catch ordinary literal dependencies in all common application syntaxes, and to make unrecognized dynamic evaluation an explicit review category rather than implying a total proof.

7.4 Performance and integration

The existing builder already performs a linear masking pass and a scan of the resulting source. The proposed token-boundary check preserves that structural cost. No extra Wolfram kernel is needed to run its mutation regressions. This is a focused build-pipeline improvement, distinct from the older general recommendation to add native mathematical CI.[15, 17, 5]

The patch tool edits only `validation/build_standalone.py`, when N03 is selected. It does not regenerate the standalone distribution automatically. That keeps an executable source change separate from artifact production and allows the existing builder tests plus the new mutation tests to be reviewed before publishing a new generated file.

8 N04: native-status classification should describe what it knows

The native wrapper computes its status using a syntactic search:

```
If[FreeQ[result, _System`Series | _System`Asymptotic],
  "Computed", "Unresolved"]
```

This detects occurrences of those heads anywhere in the result tree. It does not distinguish an unresolved active computation from a native call stored as held user data.[3]

A minimal public characterization candidate compares native and wrapped expansion of an expression constant in x :

```
Series[HoldComplete[Series[t, {t, 0, 1}]], {x, 0, 1}]

AsymptoticExpansion[
  HoldComplete[Series[t, {t, 0, 1}]], {x, 0, 1},
  "Backend" -> "Series"]
```

If the native operation returns the held constant unchanged, the wrapper's classifier labels it "Unresolved" because the `Series` stored inside `HoldComplete` is present. That stored expression is data, not a failed attempt by the outer backend to expand with respect to x . The exact public native behavior was not observed here; the script records both raw results to settle it without assuming the expected native simplification.

The current development document is careful not to equate a computed native result with a package analytic proof. Nevertheless, the property name invites a stronger reading than this syntactic predicate supports. The immediate issue is misleading status or future fallback decisions, not an established numerical wrong answer.[4]

A conservative design separates three facts: the backend returned a value; residual native-call syntax occurs at specified positions; and the system has or has not established that those occurrences represent active unresolved work. For opaque held data, "ReturnedOpaque" or an explicitly syntactic property is more accurate than a confident failure classification. A traversal must not release held bodies merely to classify them. If the implementation cannot safely interpret an arbitrary head's evaluation behavior, an unknown status is preferable to executing user data.

No code patch for this diagnostic policy is included. The raw `NativeResult` should remain unchanged, and the existing analytic/native contract barrier should remain in force. The supplied characterization is intended to establish the exact returned shape before choosing an API-compatible status migration.

9 Focused performance, API, and development conclusions

9.1 Measure complete semantic jobs

A benchmark comparing a package analytic object with `Normal[Series[...]]` can compare different jobs: branch proof and metadata construction on one side, coefficient generation with erased order information on the other. Conversely, measuring only an internal coefficient recurrence can conceal facade and source normalization costs. The new regression work should therefore separate request resolution, source admission, coefficient construction, and optional result views. This is a measurement protocol, not a native speed claim.[2, 3]

The supplied comparison runner records a single elapsed time per bounded call, its messages, head, finite expression representation, and remainder fields. Those timings are observations to inspect, not published performance rankings. Only the explicitly aligned ordinary/logarithmic forward pairs have matching algebraic cutoffs. The inverse and special-function pairs are scope probes; after a native run, their actual branches and orders must be compared before any timing summary is meaningful.

The two new concrete cost conclusions are limited and testable. First, the builder repair retains a linear lexical pass. Second, ordinary finite Fourier partial sums of the winding coefficient require an order-dependent number of harmonics to preserve the cubic uniform inverse bound. Neither conclusion recycles the older general enumeration or provenance-size findings.

9.2 Source identities need local proof obligations

The N01 patch is small, but the development lesson is precise. A normalization rule should declare its sufficient conditions at the point where it changes the source, rather than relying on the realness of the final polynomial. For the identified rule those conditions are $u > 0$, $k \in \mathbb{R}$, and, when present, $c > 0$.

A practical follow-up is a small registry of branch-sensitive identities used by source normalizers, each with a positive control, a negative control, and a boundary/winding family. The first family can vary $a^2 = -c^2$ for positive rational c , use even powers of the logarithm so the source remains real, and sample exact winding points $x_n = e^{-2\pi n/c}$. This is much more targeted than adding random expression tests and hoping branch effects appear.

Proof caching, if later added for these guards, must key on the expression, parameter assumptions, and chart. A proof obtained for a real exponent cannot be reused merely because a later normalized coefficient looks identical. This is a design consequence of the counterexample, not a claim that the current repository contains such a cache defect.

9.3 Backend defaults need one owner and one evaluation

The first milestone is the N02 repair with its explicit and delayed-default controls. The second is a documented alias ownership policy. The third is an origin-aware effective request record that distinguishes caller intent from inherited defaults. These milestones are small enough to validate independently of the broad, explicitly unfinished native-superset requirement.[3, 4]

In particular, completion of native fallback coverage should not be used to postpone a deterministic option-protocol repair. Nor should this repair be sold as completion of B01: accepting all syntax that either native backend accepts, selecting a second backend after an unresolved first attempt, and preserving arbitrary evaluation-context effects are larger obligations already discussed in the repository's compatibility document.

9.4 Keep new functionality tied to an explicit theorem

The bounded periodic inverse of Section 5 is a useful experimental extension because its assumptions, approximation, and error constant are explicit. A first implementation can remain outside the main package and return an ordinary expression plus a conditional bound. The supplied `WindingAwareReference.wl` does exactly that at the formula level; it has not been executed in a native kernel.

Only after that reference behavior is validated should it be integrated with a new coefficient representation. The first package contract should not claim arbitrary-order differentiability, because the coefficient has corners. It should claim only the operations justified by its value and Lipschitz bounds. This avoids recreating the original error under a more general object head.

10 Validation record and how to use the artifacts

10.1 Executed work

The independent test suite passed eleven tests. Its subcases include the exact retrieved builder's ten dependency forms, eight baseline misses, the proposed gate's rejections, inert and unrelated-symbol controls, companion inlining, patch-anchor refusal behavior, principal-log evaluation, the divergence formula, Lipschitz samples, and sixty numerical checks of the constructive inverse bound. The source identity of the local builder fixture was checked against the Git blob identifier returned by the connector. No checksum file is needed for this in-memory verification.

The exact mathematical conclusion for N01 is Proposition 4.1 and equation (10). The constructive error bound is Proposition 5.1. Floating-point checks exercise formulas and reference code; they are not certificate transcripts.

The actual Python output is stored in `results/python_tests.txt` and `results/independent_evidence.json`. Native status, novelty, and coverage are separate files. A missing native transcript is not represented by a fabricated empty "success" report.

10.2 Proposed source edits

`patches/apply_focused_patches.py` defaults to a unified-diff dry run. It requires the pinned Git revision, compares targeted canonical files with `git show`, requires unique source anchors, and refuses pre-existing backups. It preflights all selected edits before changing any file. The optional `--apply` mode writes only the selected canonical sources.

Its scope is deliberately limited: N01 guards the two depth-parser rules; N02 consumes the canonical backend default; N03 replaces the dependency pattern. N04 remains a design proposal. The tool does not alter the remote repository, create a commit, regenerate the root distribution, or claim that a patch has passed native validation.

The builder edit was actually executed on the local retrieved source fixture. The patch-anchor safeguards were tested on synthetic anchors for the Wolfram edits. Those tests do not establish successful loading or correct execution of a fully patched Wolfram package.

10.3 Native execution supplied for a later local run

The nine `focused_regressions.wlt` tests describe desired behavior for N01 and N02. On the unmodified baseline, several are expected to fail. The fresh-kernel runner loads the local

modular package, checks for nine returned results, and records the report. It does not execute the full upstream suite. The runner handles a documented modern result-list property and a legacy alternative, rejecting an unknown report shape rather than counting zero tests as success.[34]

The fourteen characterization/comparison cases separately localize N01's normalizer, core constructor, and public facade; compare held native data for N04; and exercise selected package/native pairs. Each call is bounded and the raw expressions and messages are recorded. The reference revision in the output is explicitly distinguished from a caller-supplied checkout that may contain proposed edits.

10.4 Practical sequence

First reproduce N01 at the private normalizer and core-constructor boundaries, then at the public facade. Check that the patch changes unsupported complex exponents into a refusal while retaining real-exponent inputs. Next run the backend-default controls, including delayed defaults and explicit overrides. Independently, run the builder mutation tests and its ordinary inlining control. Finally, inspect N04's actual native return before changing its status policy.

This sequence prioritizes source equality, deterministic API behavior, and the specific build invariant. It is not another copy of the old general release-CI roadmap. Existing project validation remains relevant, but no additional native validation is implied by the Python results in this archive.

11 Conclusion

The repository's distinguishing value is the ability to carry a source equation, a selected real approach, a structured asymptotic representation, and explicit evidence together. The current implementation has made meaningful progress in separating those contracts from opaque native delegation. The new audit findings are correspondingly concentrated at boundaries: a source rewrite that is not an identity, defaults not consulted during contract selection, a syntactically incomplete distribution gate, and an over-interpretable native status label.

The immediate mathematical repair is small: require real exponents before unwrapping the identified logarithms. The deeper constructive result is not to ban every complex intermediate, but to retain the information it carries when the overall source is real. The bounded winding coefficient supplies an explicit example: its correct inverse approximation and uniform cubic error bound can be written down and independently checked.

The recommendation is therefore to land narrowly evidenced boundary repairs, validate them with the supplied targeted native probes, and develop extensions only with explicit source identities and evidence contracts. A longer function list is less valuable than a reliable explanation of which function a returned expansion actually approximates.

A Novelty crosswalk

New ID	Closest prior item	Distinct contribution
N01	C07; C16	The coefficients become real after an invalid logarithm rewrite; an explicit real monotone source and exact inverse-value sequence disprove the resulting remainder. The issue is source identity, not only codomain admission or opaque Taylor data.
N02	D01; C09; B01	A newly inspected facade declares a configurable backend yet substitutes a literal default instead of consulting its option table. This is distinct from an explicit coefficient option being shadowed by stored model data.
N03	V01; V02	The actual Python assembler accepts eight alternative dependency-call forms. The remedy and tests concern a specific offline build invariant, not generic missing native CI or test inventory.
N04	B01; D02	A concrete all-tree native-call scan cannot distinguish held user data from an unresolved active call. No wrong asymptotic coefficient is asserted.

The following inventories were read in addition to the consolidated register. Their complete pinned paths are in `results/source_coverage.json`.

Review	Inventory read
1	evidence/findings.csv
2	README finding table and evidence scope
3	README principal findings and evidence scope
4	evidence/findings.json
5	evidence/findings.json
6	evidence/findings.csv
7	evidence/findings.csv
8	evidence/findings.csv
9	README finding table and evidence scope
10	evidence/review_crosswalk.json
11	evidence/findings-delta.json
12	evidence/novelty_ledger.csv
13	README detailed principal findings and evidence scope
14	evidence/findings.json
15	evidence/findings.csv
16	evidence/findings.csv
17	evidence/novelty_matrix.json
18	evidence/novelty_ledger.json

This screening does not claim literal full-text coverage of every old article, annex, and script. Absence from the inventories is supporting evidence of non-duplication, combined with the mechanism-level comparisons above.

B Source-review coverage

Detailed source inspection covered the ordinary core’s public definitions, exactness checks, model admission, inverse coefficient formulas, symbolic-depth parser, constructor, object assembly, and selected arithmetic entry points; the native compatibility module; inverse-function facade plumbing; the interval certificate implementation; the Gamma forward implementation; the Dirichlet special-function implementation; the structured native special-function importer; and substantial special-inverse adapter code.[2, 3, 8, 9, 14, 10, 12, 13, 11]

The root README, kernel module map, compatibility document, review register, review index, builder source, builder tests, and builder workflow were also read. Many additional specialized modules were assessed through those documents, callers, and exported contracts rather than exhaustively line by line. No uninspected implementation is certified free of defects.

The local executable fixture is only the 4,921-byte upstream Python builder. The rest of the repository was not reconstructed as an executable checkout. The supplied source manifest records the distinction, and no full-package loadability result is inferred from source inspection.

C Artifact index

Path	Purpose
article/asymptotic-current-audit.tex	Self-contained editable article.
article/asymptotic-current-audit.pdf	Compiled and visually inspected PDF.
code/independent_checks.py	Executed mathematics and actual-builder fixture tests.
code/focused_regressions.wlt	Nine unexecuted desired-contract Wolfram tests.
code/run_native.wls	Local modular-package test runner.
code/characterize_and_compare.wls	Fourteen unexecuted native characterization/comparison cases.
code/WindingAwareReference.wl	Formula-level constructive reference and conditional bound; native-unexecuted.
fixtures/build_standalone_upstream.py	Exact retrieved upstream builder.
patches/apply_focused_patches.py	Dry-run-first, pinned-checkout edit proposals for N01–N03.
results/independent_evidence.json	Actual independent results and mutation outcomes.
results/python_tests.txt	Actual eleven-test pass log.
results/native_status.json	Explicit native execution boundary.
results/novelty_ledger.json	New/old issue crosswalk.
results/source_coverage.json	Pinned source and prior-inventory coverage.

References

- [1] Vladimir Reshetnikov, *Repository README*. Repository source at the pinned revision.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/README.md>
- [2] Vladimir Reshetnikov, *Core package: public entry points, models, parseFinite (lines 916–928), construct, and object representation*. Repository source at the pinned revision.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/src/Kernel/AsymptoticAnalysis.wl>
- [3] Vladimir Reshetnikov, *NativeCompatibility: option table and dispatch (lines 5–80); native-result assembly*. Repository source at the pinned revision.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/src/Kernel/NativeCompatibility.wl>
- [4] Vladimir Reshetnikov, *Native compatibility design and limitations*. Repository source at the pinned revision.
https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/docs/development/NATIVE_COMPATIBILITY.md
- [5] Vladimir Reshetnikov, *Consolidated code-review status register*. Repository source at the pinned revision.
https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/docs/development/CODE_REVIEW_STATUS.md
- [6] Vladimir Reshetnikov, *External review index*. Repository source at the pinned revision.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/external-reports/code-review/README.md>
- [7] Vladimir Reshetnikov, *Kernel module map*. Repository source at the pinned revision.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/src/Kernel/README.md>
- [8] Vladimir Reshetnikov, *InverseFunctionExpressions: public facade and inverse-expression composition*. Repository source at the pinned revision.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/src/Kernel/InverseFunctionExpressions.wl>
- [9] Vladimir Reshetnikov, *SeriesArithmetic: arithmetic/evaluation entry points*. Repository source at the pinned revision.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/src/Kernel/SeriesArithmetic.wl>
- [10] Vladimir Reshetnikov, *GammaForward: logarithmic normalization and expansion*. Repository source at the pinned revision.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/src/Kernel/GammaForward.wl>
- [11] Vladimir Reshetnikov, *SpecialFunctionAdapters: special inverse models and bounds*. Repository source at the pinned revision.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/src/Kernel/SpecialFunctionAdapters.wl>
- [12] Vladimir Reshetnikov, *DirichletSpecialFunctions: Zeta and Lerch constructions*. Repository source at the pinned revision.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/src/Kernel/DirichletSpecialFunctions.wl>

- [13] Vladimir Reshetnikov, *NativeSpecialFunctions: structured native import*. Repository source at the pinned revision.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/src/Kernel/NativeSpecialFunctions.wl>
- [14] Vladimir Reshetnikov, *InverseCertificates: interval primitives and adaptive certification*. Repository source at the pinned revision.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/src/Kernel/InverseCertificates.wl>
- [15] Vladimir Reshetnikov, *Standalone package builder*. Repository source at the pinned revision.
https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/validation/build_standalone.py
- [16] Vladimir Reshetnikov, *Standalone builder tests*. Repository source at the pinned revision.
https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/validation/test_standalone.py
- [17] Vladimir Reshetnikov, *Standalone freshness workflow*. Repository source at the pinned revision.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/.github/workflows/standalone.yml>
- [18] Vladimir Reshetnikov, *Review 11: finding-delta ledger*. Repository source at the pinned revision.
https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/external-reports/code-review/wave-2/code-review-11/evidence/finding_s-delta.json
- [19] Vladimir Reshetnikov, *Review 17: novelty matrix, including prior native Zeta overlap*. Repository source at the pinned revision.
https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/external-reports/code-review/wave-2/code-review-17/evidence/novelty_matrix.json
- [20] Vladimir Reshetnikov, *Review 18: novelty ledger*. Repository source at the pinned revision.
https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/external-reports/code-review/wave-2/code-review-18/evidence/novelty_ledger.json
- [21] Wolfram Research, *Series*, Wolfram Language Documentation. Accessed 9 September 2026.
<https://reference.wolfram.com/language/ref/Series.html>
- [22] Wolfram Research, *SeriesData*, Wolfram Language Documentation. Accessed 9 September 2026.
<https://reference.wolfram.com/language/ref/SeriesData.html>
- [23] Wolfram Research, *InverseSeries*, Wolfram Language Documentation. Accessed 9 September 2026.
<https://reference.wolfram.com/language/ref/InverseSeries.html>
- [24] Wolfram Research, *ComposeSeries*, Wolfram Language Documentation. Accessed 9 September 2026.
<https://reference.wolfram.com/language/ref/ComposeSeries.html>
- [25] Wolfram Research, *Asymptotic*, Wolfram Language Documentation. Accessed 9 September 2026.
<https://reference.wolfram.com/language/ref/Asymptotic.html>
- [26] Wolfram Research, *AsymptoticSolve*, Wolfram Language Documentation. Accessed 9 September 2026.
<https://reference.wolfram.com/language/ref/AsymptoticSolve.html>

- [27] Wolfram Research, *AsymptoticIntegrate*, Wolfram Language Documentation. Accessed 9 September 2026.
<https://reference.wolfram.com/language/ref/AsymptoticIntegrate.html>
- [28] Wolfram Research, *AsymptoticDSolveValue*, Wolfram Language Documentation. Accessed 9 September 2026.
<https://reference.wolfram.com/language/ref/AsymptoticDSolveValue.html>
- [29] Wolfram Research, *PowerExpand*, Wolfram Language Documentation. Accessed 9 September 2026.
<https://reference.wolfram.com/language/ref/PowerExpand.html>
- [30] Wolfram Research, *Log*, Wolfram Language Documentation. Accessed 9 September 2026.
<https://reference.wolfram.com/language/ref/Log.html>
- [31] Wolfram Research, *SetOptions*, Wolfram Language Documentation. Accessed 9 September 2026.
<https://reference.wolfram.com/language/ref/SetOptions.html>
- [32] Wolfram Research, *Expressions: special ways to input expressions*, Wolfram Language Documentation. Accessed 9 September 2026.
<https://reference.wolfram.com/language/tutorial/Expressions.html>
- [33] Wolfram Research, *Apply*, Wolfram Language Documentation. Accessed 9 September 2026.
<https://reference.wolfram.com/language/ref/Apply.html>
- [34] Wolfram Research, *TestReport*, Wolfram Language Documentation. Accessed 9 September 2026.
<https://reference.wolfram.com/language/ref/TestReport.html>